Attribute Trouble

Object attributes (which I describe in *Mimic Methods*, on page 217) contain a hidden trap for the unsuspecting programmer:

common_idioms/attribute_trouble.rb

```
class MyClass
  attr_accessor :my_attribute

  def set_attribute(n)
    my_attribute = n
  end
end

obj = MyClass.new
obj.set_attribute 10
obj.my_attribute          # => nil
```

This result is probably not what you expected. The problem is that the code in set_attribute is ambiguous. Ruby has no way of knowing whether this code is an assignment to a local variable called my_attribute or a call to a *Mimic Method (218)* called my_attribute=. When in doubt, Ruby defaults to the first option. It defines a variable called my_attribute, which immediately falls out of scope.

To steer clear of this problem, use self explicitly when you assign to an attribute of the current object. Continuing from the previous example:

```
class MyClass
  def set_attribute(n)
    self.my_attribute = n
  end
end

obj.set_attribute 10
obj.my_attribute          # => 10
```

If you're a jaded Ruby expert, you might ask yourself a subtle question that completely escaped me while writing the first edition of this book. What if MyClass#my_attribute= happens to be private? In *What private Really Means*, on page 35, I said that you cannot call a private method with an explicit self receiver—so it seems that you're out of luck in this (exceedingly rare) case. The answer to this conundrum is one of Ruby's few ad-hoc exceptions. Attribute setters such as my_attribute= can be called with self even if they're private:

```
class MyClass
  private :my_attribute
end

obj.set_attribute 11      # No error!
obj.send :my_attribute    # => 11
```